ENTERPRISE DIGITAL ENGINE

MaargSetu

EDE PLATFORM · ARCHITECTURE APPROACH

Logistics Vertical — Three-Domain Structure

One shared base domain plus two product domains — freight forwarding and NVOCC — so each product can be sold, activated and released on its own, over one set of shared masters.

> **Approach:** `logistics_base` holds the common modules; `logistics_freight_forwarding` and `logistics_nvocc` depend on it and never on each other. Verified to work with **zero framework change**.

> **Two things decide the outcome:** the module dependency graph — not preference — fixes what can move to the base; and the tier rule needs a **boot guard**, because the loader accepts peer coupling silently.

PREPARED FOR
Engineering Lead · Product Owner

SOURCE INPUTS
SOW — BRD N1 · live codebase audit · loader probes

DATE
30 July 2026

SECTION 01

01 The structure

Three domain packages. Common modules live in the base; each product domain holds only what is genuinely its own. The two products never reference each other — that is the property the whole structure exists to protect.

BASE DOMAIN **logistics_base** *What both products need. Depends only on the platform.*	masters · equipment_control · pricing · sales_crm · booking · shipments · operations_execution · profitability
PRODUCT DOMAIN **logistics_freight_forwarding** *Freight forwarding only.*	consolidation · documentation · tracking
PRODUCT DOMAIN **logistics_nvocc** *NVOCC / shipping agency only.*	agency (new)

Dependencies point upward only — products read from the base, the base never reads back, and the two products never see each other.

Activation is the sales lever

```
ACTIVE_DOMAINS = ["logistics_base", "logistics_freight_forwarding", "logistics_nvocc"]   # full service
ACTIVE_DOMAINS = ["logistics_base", "logistics_freight_forwarding"]                      # forwarding only
ACTIVE_DOMAINS = ["logistics_base", "logistics_nvocc"]                                   # NVOCC only
```

This needs no framework change — verified

App keys derive from folder position, so the packages read exactly as intended: `logistics_base.masters`, `logistics_freight_forwarding.tracking`, `logistics_nvocc.agency`. Probing the loader's dependency validator directly:

Case	Result
Product domain → base domain, base loaded first	ACCEPTED — the structure works today
Same dependency, base **not** loaded yet (wrong order)	REJECTED — with a message naming the missing key; ordering is self-enforcing
Product domain → **peer** product domain	ACCEPTED — the loader does **not** block it

> **⚠ The third row is the whole risk**
> The structure is a **packaging and governance** change, not a kernel change — nothing needs building to make it boot. But nothing stops a developer wiring one product to the other later, and that mistake would land in `main` with a green build. Section 02 closes it.

Naming

Use **`logistics_base`**, not `logistics_foundation` — "foundation" already names the platform layer, and the second name would read as if the domain lived there. The underscore in the app key is correct: the no-underscore rule governs *model* keys; app keys derive from folder names, which are snake_case.

SECTION 02

02 The tier rule, and the guard that enforces it

The governing rule today states it absolutely: *domains never depend on other domains.* This structure needs that re-expressed, because a product domain depending on the base is a domain-to-domain edge by the letter of it. **The invariant that actually matters is unchanged: peer products must never couple.**

Reformulate around tiers

Tier	May depend on	Must never depend on
Base domain `logistics_base`	The platform layer only	Any product domain
Product domain `logistics_freight_forwarding`, `logistics_nvocc`	The platform layer · any **base** domain · its own sibling modules	Another **product** domain

That keeps the graph acyclic and one-directional, keeps each product independently activatable, and states plainly the one edge that is now legal — a product domain reading from the base.

Make it a guard, not a comment

Everything the check needs is already in the registry — each app record carries its domain and its declared dependencies. Declare the tiers, then validate at boot beside the existing validators:

```
ACTIVE_DOMAINS = ["logistics_base", "logistics_freight_forwarding", "logistics_nvocc"]
BASE_DOMAINS   = ["logistics_base"]      # every other active domain is a product domain
```

Case	Verdict
Dependency is a platform module	ALLOW
Dependency is a sibling module in the same domain	ALLOW
Product domain depends on a base domain	ALLOW
Base domain depends on a product domain	FAIL
Product domain depends on a different product domain	FAIL

Cheap, and it follows an established pattern

Around 25 lines, sitting alongside the boot validators that already check model extensions, delegation declarations and view components. It turns the tier rule from documentation into something a mistake cannot pass — and it is the single change that makes this structure safe to hand to a team.

Governance change required

The naming and placement rules need amending in one change before the first module moves: the tier rule above, the prefix convention in §04, and the placement test rewritten to ask *"which tier?"* rather than *"platform or domain?"*. Without it, the very first review reads the new layout as a violation.

SECTION 03

03 What goes where — the graph decides

The partition is not a matter of preference. Every module's declared dependencies were resolved transitively: **putting a module in the base forces its entire dependency closure into the base with it.**

11	4	8	3
MODULES TODAY	REVERSE-LEAVES	MOVE TO BASE	STAY WITH FREIGHT

Only four modules can stay behind

A module can remain in a product domain only if nothing in the base depends on it. Exactly four qualify — `documentation`, `operations_execution`, `profitability`, `tracking`. Every other module is depended upon by something else.

Module	Depended on by	Putting it in the base also forces…
masters	all 10 others	nothing — safe leaf, moves first
equipment_control	booking, consolidation, documentation, operations_execution, shipments, tracking	masters
shipments	consolidation, documentation, operations_execution, profitability, tracking	booking, equipment_control, masters, pricing, sales_crm
pricing	booking, profitability, sales_crm	masters
sales_crm	booking, profitability, shipments	masters, pricing
booking	profitability, shipments	equipment_control, masters, pricing, sales_crm
consolidation	documentation, tracking	booking, equipment_control, masters, pricing, sales_crm, shipments
documentation · operations_execution · profitability · tracking	`nothing`	the six or seven above, depending on the module

The resulting cut

Domain	Modules
`logistics_base` — 8	masters · equipment_control · pricing · sales_crm · booking · shipments · operations_execution · profitability
`logistics_freight_forwarding` — 3	consolidation · documentation · tracking
`logistics_nvocc` — 1 new	agency — *SOW Phase 2 is base work; see below*

Why those three stay. `documentation` and `tracking` both depend on `consolidation`, whose co-load / consol model is freight-forwarding-specific — so all three move together or not at all. Keeping them out also matches the SOW, where bill-of-lading, manifest and container-tracking execution are separate downstream specifications, not this scope. Verified tier-valid: nothing in the base depends on any of the three.

> **⚠ SOW Phase 2 is base work, not NVOCC work**
>
> The SOW's Phase 2 is titled *"Container & Equipment Control"* (D2.1–D2.8), which reads like an NVOCC module — it is not. Every one of those deliverables targets a **base-domain** model: the container type master, the equipment registry, the status masters, the movement-event types, and the facility and trade-location masters. The freight product already ships equipment tracking today, and ISO check digits, ownership types and CODECO/COARRI codes are container concerns, not agency concerns. **NVOCC owns exactly one slice:** the principal link on the equipment record, contributed by extension from the agency module — because it references an NVOCC model, and the base can never point at a product domain.

> **Default to the base when unsure**
>
> The two mistakes are not symmetric. A module in the base that only one product uses is harmless over-inclusion. A module in a product domain that the *other* product later needs leaves you blocked — or tempted into exactly the peer dependency the tier rule forbids. If NVOCC later needs shared documentation or tracking, moving all three across is a follow-up of the same shape.

SECTION 04

04 Model keys stay exactly as they are

This is the decision that makes the restructure affordable. **Model keys and app keys are independent** — the key is declared in the decorator, the app key derives from folder position. Moving a module changes the app key and nothing about model identity.

42	284	72	0
MASTERS MOVING	EXTERNAL REFERENCES	FILES TOUCHING THEM	KEYS RENAMED

Keeping the keys preserves all 284 references, all 32 seed files — including the CSVs, whose **filenames must exactly match the target model key** — every view, every stored report definition, and the on-demand trade-location sync, which resolves its target by key.

Prefix convention across the tiers

Prefix	Owner	Meaning
`res.*` / `ir.*`	Platform	Universal resources · framework metadata
`logistics.*`	**`logistics_base`**	Shared across the logistics vertical
`mixin.*`	`logistics_base`	Keyed abstract shapes — see §06
`logistics_ff.*`	`logistics_freight_forwarding`	Freight-specific — **new models only**
`nvocc.*`	`logistics_nvocc`	NVOCC / agency-specific

Existing keys are grandfathered

`logistics.shipment`, `logistics.booking`, `pricing.rate` and `crm.lead` keep their keys even where they sit in a product domain. Renaming them means a table rename plus foreign-key repointing across roughly ten child models, plus invalidation of every stored reference naming the old key — for zero capability. Models written from here on take the new tier prefix, so the prefix tells you the tier for all future work.

What proves a move changed nothing

Every relocation must be verified by running the migration generator against the moved module and confirming the output is **empty**. That is the only mechanical proof that the move changed layout and not schema. A non-empty diff means something was edited in transit — fix the model and regenerate rather than accepting the revision.

⚠ No implicit behaviour change either

The record-rule and organization-scope bypass lists are explicit model keys plus framework prefixes — there is no blanket rule keyed on `res.*`. So neither keeping the current keys nor renaming them would silently alter row-level filtering. Checked rather than assumed.

SECTION 05

05 What moves into the shared masters

All 42 masters move wholesale. Each was audited against both businesses, and every one is needed by both — which is itself the confirmation that this module was always the shared layer.

Group	Models	Needed by NVOCC because
Trade locations	unlocode.master	Ports of load and discharge on every bill of lading and manifest
Facilities	facility.master, facility.zone.master, facility.type.master, facility.zone.type.master, chl.type.master	Yard, depot, CFS, terminal and inland container depot control
Equipment	transport.mode, shipment.type, load.type, equipment.category / subcategory / identifier.type / type	Container type classification and ISO mapping
Operational states	equipment.status, equipment.condition, usage.status, movement.event.type	Container status lifecycle and movement-event mapping
Maintenance	maintenance.type / status, inspection.type / result.status	Repair yard, under-repair state, equipment interchange inspection
Security	seal.type, seal.change.reason	Seal control on gate-in and gate-out
Documents	document.type, reference.context.type	Bill of lading, manifest and delivery-order document typing
Products & trade	incoterm.master, product.master, commodity.master, trade.lane	Commodity and lane on every booking
Cargo	package.type, haz.class	Packing and dangerous-goods declaration
Parties	carrier, carrier.agent, provider	The principal's carrier and agent network
Vessels	vessel, vessel.category	Vessel call, manifest and voyage reference
Rules & misc	service.mode, volumetric.rule, lane.restriction, sailing.schedule, vendor.service, tag	Chargeable weight, routing restrictions, schedules, vendor roles

Also build here — a gap NVOCC surfaced

There is **no charge-code master anywhere today** — only transactional charge lines on rates, bookings, shipments and documents. It belongs in the base, where pricing, booking, shipments and documentation can all use it. Building it inside an NVOCC module instead would make four existing modules depend on NVOCC.

One container registry, not two

Equipment control moves to the base as a **shared concrete model**, not a copied shape. That keeps container numbers globally unique and lets one register report cover both businesses. A second, parallel container model would split movement history across two tables and break every existing equipment report — recoverable only by migration.

SECTION 06

06 Shared shapes — the mechanism

Where both products need the same record *shape* but their own records — package dimensions and weights, charge lines, party lines — declare the shape once in the base as a keyed abstract, and let each product subclass it.

What the framework already gives you

- A declaration-only abstract base — **no table, no migration**.
- Field **and** constraint collection walk the full inheritance chain, so everything on the base flows into every concrete.
- Methods, commands and lifecycle hooks inherit normally, including calls up to the base.
- Shipped precedents already used by around 40 models — the chatter and attachment mixins.
- A model-key parameter for abstracts already exists on the model decorator.

> ⛔ **The keyed path is broken today — fix it first**
> Declaring an abstract *with a model key* sets the key and returns early without registering it. But the loader's model scan gates **only** on "does this class have a model key?" — there is no abstract guard anywhere in it — so it calls the concrete registration path, which raises `Abstract model cannot be registered`. **Verified by probe.** The flag has **zero usages** in the codebase, which is why nobody has hit it: every abstract today is declared without a decorator, so it has no key and never reaches that path.

Three small additive fixes

Where	Change
Registry	An abstract-by-key map plus registration and lookup accessors, mirroring the concrete ones.
Loader	Route classes marked abstract to that new path instead of the concrete registration.
Boot	Validate lineage in the same pass as the existing model-extension check.

Approval-gated as a kernel change, but with no effect on any existing model — nothing uses the flag today. Ship it with tests: the path is entirely uncovered.

Derive the lineage; don't declare it

The obvious next step is a parameter naming the parent by key. **Don't make that the mechanism.** Field collection walks the inheritance chain, so a key-only edge with no real inheritance would carry fields and nothing else — no methods, no commands, no hooks, no calls to the base, and no editor navigation.

Since shared *behaviour* is the entire point, keep real inheritance and **derive** the keyed lineage from it: at registration, walk the inheritance chain and record the key of every abstract base. The registry can then answer *"which models inherit this shape?"* for documentation, the admin surface and review — with zero authoring burden and **no possibility of drift**, because the lineage is read from the truth rather than restated beside it.

SECTION 07

07 Shared shapes — using them

The model key is for introspection. The inheritance itself is wired by an ordinary import, legal because the product domain declares the base domain as a dependency.

```
@api.model("mixin.package.measure", abstract=True)
class PackageMeasureMixin(AbstractDomainModel):
    """Dimensions, weights, volume. No table — contributes columns to each consumer."""
    length          = fields.Decimal(precision=12, scale=3)
    gross_weight    = fields.Decimal(precision=14, scale=4)
    dimension_uom_id = fields.Reference("res.uom", on_delete="set null")
    def chargeable_weight(self, ratio): ...        # shared behaviour, declared once

# ── consumed from a product domain: the freight-forwarding manifest line ──
@api.model("consolidation.manifest.line")
class ManifestLine(PackageMeasureMixin):
    console_id  = fields.Reference("consolidation.console", on_delete="cascade", required=True)
    package_qty = fields.Integer()
```

Numerics are Decimal or Monetary — there is no Float field type. Export the mixin from a stable path and import it in the owning module so it registers even when no consumer is active.

Composition and collisions — verified

- Mixin fields merge into the concrete's field set; the concrete's own declaration overrides any mixin's.
- On a field-name collision between two mixins, **the first base in the list wins — silently.** Coordinate or prefix names across mixins.
- Methods inherit normally, including calls up to the base.

"Extending" a shape — three different cases

You want to…	Do this	Not this
Add product-specific fields alongside the shape's	Declare them on the **concrete**	—
Add a field **every** consumer needs	Edit the mixin in the base — you own that module	The extension SDK — it cannot target an abstract
Add a field to a **concrete** model owned by another module	The extension SDK, with your own migration	Editing their model file

Pick the right flavour — the FK question decides

Flavour	Tables	FK to the shared type?	Behaviour inherited?
Abstract mixin `default`	one per concrete	No	Yes — full inheritance
Shared concrete	one, shared	Yes	n/a — it is the model
Delegation	parent + child	Yes	Partial — data only, not methods

> **Mixin for shapes; shared concrete for registries**
> A mixin **copies columns into each consumer's own table**. That is right for a repeating shape. It is wrong for a shared registry — two tables means identity is not unique and no single query spans both. Where something must reference "either kind" of a per-product record, an abstract cannot help: a foreign key cannot target one. Use a shared concrete parent with delegation, and pair it with an abstract base for the behaviour.

SECTION 08

08 Execution plan

Nine work streams in four buckets, dependency-ordered. Every relocation is proven by an empty generated migration; every phase is verified against all three activation combinations.

#	Work stream	Bucket	Gate
1	Domain tier guard	Platform	Approval — kernel change. Blocks everything after it.
2	Keyed abstract registration	Platform	Approval — kernel change. Blocks the shared shapes.
3	Base domain + masters move	Migration	Empty generated migration
4	Core modules move — 5, in dependency order	Migration	Empty migration after **each** module
5	Engine modules move — 2	Migration	Empty migration; tier guard passes
6	Rename to freight forwarding	Rename	Single revertible commit
7	Shared shapes (mixins)	Migration	Empty migration after extraction
8	NVOCC domain + agency module	NVOCC	NVOCC-only tenant boots
9	Container control — SOW Phase 2	Migration *(base, not NVOCC)*	No second container model anywhere

Sequencing

Approve streams 1 and 2 → build them → move masters → move the five core modules in dependency order → move the two engines → rename the freight domain → extract the shared shapes → extend the base equipment registry for container control, and build the NVOCC agency module.

Work that cannot start yet

Item	Blocker	What must land first
Principal-wise data scoping	Approval-gated kernel change	The platform has a mature scoping mechanism for exactly one dimension — it injects an ownership filter on every read and fails closed. Principal is a second, orthogonal dimension. Generalise that mechanism, or accept a weaker rule-based fallback that still needs an extension point.
Tax and withholding setup	Localization module not started	Tax is finance-wide, not logistics-specific — platform-layer work. Sequence it ahead, or move the deliverable out of scope.
Regional control framework **India & UAE packs**	Country-scope evaluator not implemented	The per-field country-scope slot exists on every extension field, but **nothing reads it**. Hard prerequisite for both country packs.
Finance-mapping reference	Layer decision open	Every domain hands off to external accounting, so the mapping is platform-layer. Decide its home against the standing decision that this platform holds no accounting domain.

SECTION 09

09 What to do

#	Rule	Why
1	**List the base domain first** in the active-domain list	Dependency validation requires a dependency to be already loaded, and domain load order is settings order.
2	Name it `logistics_base`, not `logistics_foundation`	"Foundation" already names the platform layer; the alternative reads as if the domain lived there.
3	Express the rule as **tiers** and **enforce it at boot**	The loader accepts peer coupling silently — verified. A guard turns the rule into something a mistake cannot pass.
4	**Keep model keys unchanged** when modules move	Preserves 284 references, 32 seed files whose CSV names encode keys, every view, every report definition, and the location sync.
5	Let the **dependency closure** decide the partition	Putting a module in the base forces its whole closure with it. Only four modules can stay behind.
6	**Default an ambiguous module to the base**	Over-inclusion is harmless; a product module the other product needs leaves you blocked or tempted into peer coupling.
7	Move the masters **wholesale**	All 42 are needed by both. Push a genuinely single-product master back down later.
8	Keep equipment control as one **shared concrete**	Globally unique container numbers and one register report across both businesses.
9	**Mixins for shapes; shared concretes for registries**	A mixin copies columns per table — two tables means identity is not unique and no query spans both.
10	Fix the three keyed-abstract gaps **before** any mixin ships	The path is declarable but crashes at boot, and has zero test coverage.
11	**Derive** the keyed lineage from real inheritance	Registry-queryable lineage, full method inheritance, and drift made impossible.
12	Extract shapes **from** existing models, keeping field sets identical	Turns a shared-shape change into a no-op migration instead of a schema change.
13	Prove every move with an **empty generated migration**	The only mechanical proof that a relocation changed layout and not schema.
14	Test **three activation combinations** every phase	A tiered split is only real if each product boots on the base alone.
15	Build the shared **charge-code master** in the base	It exists nowhere today, and four modules need it.

SECTION 10

10 What not to do — structural

These change the dependency graph or the ownership of a model. Each is expensive to unwind once code has been written against it.

Don't	Why it breaks	Instead
Let the two product domains reference each other	The one edge the tier rule exists to forbid — and the loader accepts it silently, so it lands green.	Lift the shared concept into the base domain.
Let the base domain depend on a product domain	Inverts the tier and creates a cycle in everything but name.	The base depends on the platform layer only.
Ship the tier rule as documentation only	Nothing enforces it; the first mistake passes review and CI.	The boot guard, in the very first work stream.
Put the base domain after a product domain in the active list	Dependency validation fails — the dependency is not loaded yet.	Base domain first, always.
Rename model keys because a module changed domain	Keys and app keys are independent. Renaming churns 284 references and 32 seed files for zero gain.	Keys stay; new models take the new tier prefix.
Rename existing product model keys	Table rename plus foreign-key repointing across roughly ten child models, plus invalidation of every stored reference naming the old key.	Grandfather them; apply the new prefix only to new models.
Fork a second container registry for NVOCC	Container numbers stop being globally unique, movement history splits across two tables, and every existing equipment report breaks.	One shared concrete in the base; each product extends it.
Cherry-pick which masters move	Creates two masters modules and a new seam in the place hardest to change later.	Move wholesale; push anything genuinely single-product back down as a follow-up.

> ⚠ **The first two are the ones to guard hardest**
> Both boot successfully and pass every test. Nothing in the framework will tell you they happened — only the guard from §02, or review. Everything else on these two pages announces itself sooner.

SECTION 11

11 What not to do — modelling & process

Don't	Why it breaks	Instead
Give an abstract shape a table or its own migration	It stops being abstract — you get a third table that should not exist.	Declaration-only: no table, no migration.
Point a foreign key at an abstract	It cannot resolve — abstracts never enter the registered-model map.	A shared concrete parent with delegation, or keep the child in the product domain.
Use the extension SDK on an abstract	Its validator resolves concrete targets only, so it fails at boot.	Edit the mixin (you own the base), or extend each concrete.
Overload the extension decorator to mint a new model	Extend-in-place and declare-new-model have opposite migration consequences; the call site becomes unreadable.	The extension decorator extends; the model decorator declares.
Make a key-only parameter the inheritance mechanism	Fields would cross but not methods, commands, hooks or calls to the base — and behaviour was the entire point.	Real inheritance; derive the lineage. An optional key parameter only as a validated assertion.
Ship a keyed abstract before the kernel fix	Boot fails — the loader has no abstract guard and routes it to concrete registration.	Land the registry, loader and validation fixes with tests first.
Abstract a shape before its second consumer exists	A wrong shared base is harder to remove than a duplicate is to merge.	Extract when the second consumer is actually being written.
Change the kernel without approval to unblock a domain	A framework-shape change made under delivery pressure, with domain-shaped tests.	Raise it as a platform change with its own roadmap entry and tests.
Overload "principal" for the carrier identity	The runtime already uses that word for the authenticated user; access control and carrier identity would read the same slot. The failure is silent and security-relevant.	Explicitly qualified names, documented at the module boundary.
Accept a non-empty generated migration after a move	Means the relocation silently changed schema — the one failure this migration must not have.	Fix the model, regenerate, re-verify.
Hand-author a migration	Diverges from the model, breaks the next generated revision, and drifts across database engines.	Fix the model and regenerate; surface generator failures rather than working around them.
Mark a deliverable done without demo data	It cannot be demonstrated at the phase-exit review, and the next developer has no working example.	Ship the use-case document and demo records, and smoke-test them.

SECTION 12

12 Decisions to close

Seven open items. Two gate the critical path; the rest are scope questions the change-management clause exists to handle.

#	Decision	What is being asked	Owner
1	Governance amendment	Approve the tier rule and the prefix convention in one change, before the first module moves. **Blocks work stream 1.**	Eng Lead
2	Keyed-abstract kernel fix	Approve the three additive changes that make a keyed abstract usable. **Blocks the shared shapes.**	Eng Lead
3	Base domain home	Confirm the base sits as a domain package alongside the products, with the tier guard as its safety net.	Eng Lead
4	Tier calls for four modules	Booking, shipments, consolidation and how much of documentation is shared. Business semantics, not architecture — the closure analysis constrains but does not decide them.	Product Owner
5	Principal scope dimension	Generalise the existing scope mechanism into a registry of dimensions, or accept the weaker rule-based fallback and record the residual risk.	Eng Lead
6	Localization sequencing	Tax, the regional framework and the two country packs all depend on a platform module that has not started. Sequence it ahead, or descope those four deliverables.	Product Owner
7	Finance-mapping layer	Where the mapping master lives, against the standing decision that this platform holds no accounting domain.	Eng Lead + PO

⚠ Two approvals gate everything

Work streams 1 and 2 are kernel changes and need explicit approval before any edit to the framework core. Stream 1 blocks every migration stream; stream 2 blocks the shared shapes. Getting both approved early is the single highest-leverage scheduling action available.

The bottom line

The three-domain structure works today with **zero framework change** — it is packaging and governance, not engineering. What engineering it does need is small and additive: one boot guard to make the tier rule enforceable, and one fix to a registration path that nothing currently uses. Everything else is relocation, proven safe by empty migrations, and the reuse it unlocks is what lets NVOCC ship over the same masters, pricing, booking and container registry the freight product already runs on.